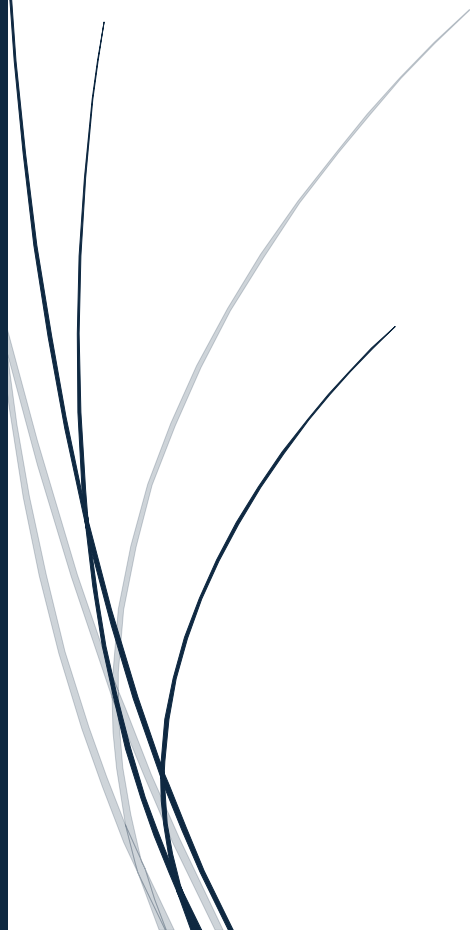




Cybersecurity 266IT

Progetto Codice Python

S2 L5



Andrea Calabretta

Sommario

INTRODUZIONE.....	2
PROGETTO CODICE PYTHON: SCOPE.....	3
Lo scopo del codice	3
ERRORI STRUTTURALI DEL CODICE	3
La struttura: la funzione “def” riga 11	3
La struttura: il “break” riga 8	4
La struttura: “if, elif comando ==” riga 12,15,18	4
ERRORI SINTATTICI.....	5
Sintassi: riga 3	5
Sintassi: riga 4	5
Sintassi: riga 13.....	5
CODICE CORRETTO	5
CONCLUSIONI	6

INTRODUZIONE

Per agire come un Hacker bisogna capire come pensare fuori dagli schemi. L'esercizio di oggi ha lo scopo di allenare l'osservazione critica. Dato il codice si richiede allo studente di:

1. Capire cosa fa il programma senza eseguirlo.
2. Individuare nel codice sorgente le casistiche non standard che il programma non gestisce (esempio, comportamenti potenziali che non sono stati contemplati).
3. Individuare eventuali errori di sintassi / logici.
4. Proporre una soluzione per ognuno di essi.

Il codice proposto nell'esercitazione è il seguente (screenshot 1):

Screenshot 1

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
```

Codice progetto

PROGETTO CODICE PYTHON: SCOPE

Lo scopo del codice

In prima istanza, come richiesto dall'esercizio, occorre capire e considerare cosa fa o meglio, **cosa dovrebbe fare il programma** senza eseguirlo. Risulta evidente come l'intenzione di questo specifico codice sia quello di restituire all'utente (comando input) una serie di informazioni riguardo:

- **La data corrente:** (datetime.datetime.today) divisa in: giorni/mesi/anni ("%d/%m/%Y") dove per "d" si intende "day" (giorno) per "m" si intende "month" (mese) e per Y si intende "year" (anno)
- **L'ora corrente:** (datetime.datetime.now.time) divisa in: ore e minuti "%H:%M" dove per "H" si intende hours (ore) e per "M" si intende minutes (minuti).
- **Assistente Virtuale:** nel caso in cui l'utente dovesse chiedere "Come ti chiami" alla macchina.
- **Uscita dal programma:** se l'utente chiede "esci" il programma **dovrebbe** interrompersi.

In questo caso, l'uso del condizionale all'interno delle frasi appena utilizzate non è casuale in quanto il codice proposto nello screenshot 1 comprende errori sia di strutturazione sia di sintassi e logica. Nel seguente capitolo si andranno dunque ad analizzare gli errori trovati all'interno del codice tentando di proporre infine una soluzione.

ERRORI STRUTTURALI DEL CODICE

La struttura: la funzione "def" riga 11

In primo luogo, ciò che mi ha colpito sin da subito è la strutturazione del codice.

"Una funzione in Python è un set di istruzioni che sarà eseguito quando la funzione verrà richiamata all'interno del programma. Le funzioni sono utili per organizzare il codice in blocchi logici riutilizzabili."¹

La funzione "def" a rigo 11 dovrebbe essere scritta prima del ciclo while. Ciò genera un errore nella corretta esecuzione dei comandi. Python, infatti, legge il codice dall'alto verso il basso e non avendo stabilito quindi una funzione prima, il ciclo while True non si attiverebbe.

¹ Presentazione slide, teoria S2 L2, Francesco Capparelli, 2026, pp.66.

La struttura: il “break” riga 8

Un altro errore strutturale all’interno del codice è il comando “break”. Tale comando che dovrebbe avere lo scopo di interrompere un ciclo in maniera condizionata, nel codice fornito dall’esercizio è **indentato in maniera errata**. A causa di questa indentazione il comando “break” è eseguito indipendentemente dalla condizione dell’if precedente, determinando quindi l’uscita immediata dal while True e compromettendo anche la corretta esecuzione del programma.

La struttura: “if, elif comando ==” riga 12,15,18

Non nego che per vedere questo errore ho avviato il programma con le modifiche apportate da me (con anche gli errori sintattici sistemati che vedremo nel capitolo successivo). All’avvio del programma, dunque, la macchina mi rispondeva sempre con “Non ho capito la tua domanda” a qualsiasi richiesta io facessi (screenshot 2).

Screenshot 2

```
>>> import datetime
...
... def assistente_virtuale(comando):
...     if comando == "Qual è la data di oggi?":
...         oggi = datetime.date.today()
...         risposta = "La tua data di oggi è" + oggi.strftime("%dd/%m/%y")
...     elif comando == "Che ore sono?":
...         ora_attuale = datetime.datetime.now().time()
...         risposta = "L'ora attuale è" + ora_attuale.strftime ("%H:%M")
...     elif comando == "Come ti chiami?":
...         risposta = "Mi chiamo Assistente Virtuale"
...     else:
...         risposta = "Non ho capito la tua domanda."
...     return risposta
... while True:
...     comando_utente = input("Cosa vuoi sapere?")
...     if comando_utente == "esci":
...         print("Arrivederci!")
...         break
...     else:
...         print(assistente_virtuale(comando_utente))
...
Cosa vuoi sapere? la ata
Non ho capito la tua domanda.
Cosa vuoi sapere? la data
Non ho capito la tua domanda.
Cosa vuoi sapere?data
Non ho capito la tua domanda.
Cosa vuoi sapere? data
Non ho capito la tua domanda.
Cosa vuoi sapere? Qual è la data di oggi
Non ho capito la tua domanda.
Cosa vuoi sapere? Qual è la data di oggi?
Non ho capito la tua domanda.
```

Errore nel codice

Ci ho messo un po’ a capirne il motivo.

In realtà, il problema non è legato al funzionamento generale del ciclo o della funzione, bensì alla logica con cui veniva interpretato l’input dell’utente.

In una prima versione del codice, infatti, il confronto tra il comando inserito e le condizioni previste avveniva tramite un’uguaglianza diretta (ad esempio comando == "Qual è la data di oggi?"). Questo approccio si è rivelato eccessivamente rigido, poiché richiedeva che l’utente digitasse esattamente la stessa stringa, inclusi maiuscole, spazi e punteggiatura.

La soluzione adottata è stata quella di rendere il sistema più flessibile intervenendo su due aspetti fondamentali. In primo luogo, ho normalizzato l'input utilizzando il metodo `lower()`, così da eliminare le differenze tra maiuscole e minuscole. In secondo luogo, ho sostituito il confronto diretto con la ricerca di parole chiave all'interno della stringa, utilizzando l'operatore "in" (ad esempio `if "data" in comando`).

ERRORI SINTATTICI

Sintassi: riga 3

In riga 3 al comando `while True` mancano i due puntini ":",.

Sintassi: riga 4

In riga 4 vi è uno spazio di troppo dopo (`Cosa vuoi sapere?.I`)

Sintassi: riga 13

Per trovare questo errore a riga 13 ho consultato la libreria scritta in riga 1 "`datetime`". Online si trova una dettagliata analisi della libreria `datetime` ed è lì che ho scoperto l'errore nel codice fornito dall'esercizio. In realtà, a riga 13 manca un punto dopo `datetime.date.today()`.

CODICE CORRETTO

Anche se l'esercizio non richiede esplicitamente la riscrittura corretta del codice, ho comunque voluto testare e provare se la correzione degli errori sia strutturale e logici sia di sintassi si è rilevata funzionale.

Il codice corretto (screenshot 3)

Screenshot 3

```
>>> import datetime
... def assistente_virtuale(comando):
...     comando = comando.lower()
...     if "data" in comando:
...         oggi = datetime.date.today()
...         return "La data di oggi è " + oggi.strftime("%d/%m/%Y")
...     elif "ora" in comando:
...         ora_attuale = datetime.datetime.now().time()
...         return "L'ora attuale è " + ora_attuale.strftime("%H:%M")
...     elif "chiami" in comando:
...         return "Mi chiamo Assistente Virtuale"
...     else:
...         return "Non ho capito la tua domanda."
... while True:
...     comando_utente = input("Cosa vuoi sapere? ")
...     if comando_utente.lower() == "esci":
...         print("Arrivederci!")
...         break
...     else:
...         print(assistente_virtuale(comando_utente))
...
Cosa vuoi sapere? la data di oggi
La data di oggi è 17/04/2026
Cosa vuoi sapere? l'ora
L'ora attuale è 12:24
Cosa vuoi sapere? Come ti chiami?
Mi chiamo Assistente Virtuale
Cosa vuoi sapere? Hai fame?
Non ho capito la tua domanda.
Cosa vuoi sapere? esci
Arrivederci!
>>>
```

Codice corretto

CONCLUSIONI

Come già accennato precedentemente, il più grande ostacolo incontrato è stato comprendere il funzionamento dell'operatore `==` applicato al confronto tra stringhe.

In particolare, la difficoltà principale risiedeva nel fatto che tale operatore richiede una corrispondenza esatta tra i valori confrontati, rendendo il programma estremamente rigido e poco adattabile alle variazioni dell'input dell'utente.

L'esercizio è stato sicuramente utile per comprendere non solo gli aspetti sintattici del linguaggio Python, ma soprattutto le implicazioni logiche legate alla gestione degli input e al flusso di esecuzione del programma.

Mi ha permesso, inoltre, di sviluppare una maggiore consapevolezza nell'individuazione degli errori, distinguendo tra problemi di sintassi e problemi di natura logica.